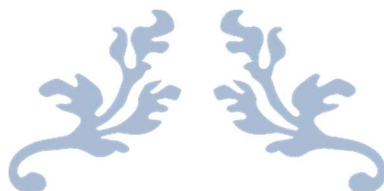




JAGRAN INSTITUTE OF MANAGEMENT

An Institute approved by AICTE, Ministry of HRD, Govt. of India & Affiliated to
Dr. A.P.J. Abdul Kalam Technical University, Lucknow, (College Code : 434)



PRACTICAL FILE ON “Software Engineering Lab” (Subject Code: BMC-352)



“In the Fulfillment of the MCA-Second Semester Curriculum”
(MCA-Batch: 2024-2026)

SUBMITTED TO:

MR. ASHISH MISHRA
(ASSISTANT PROFESSOR)
DEPARTMENT OF MCA
Jagran Institute of Management
Kanpur

SUBMITTED BY:

//////////

AKTU Roll No. 20000000
MCA-3rd Sem.
Session: 2025-26

Acknowledgment

Before I proceed further, I would like to spend some time expressing my gratitude to all those who have been involved in guiding me out during the entire curriculum.

I wish to express my most sincere gratitude and appreciation to **Mr. Ashish Mishra**, *Assistant Professor, Department of MCA, Jagran Institute of Management, Kanpur*, for giving me inspiration and requisite facility by giving me a chance to show my capabilities and also form asking me feel comfortable in the strictly professional environment of the college premises.

I am also grateful for her cooperation in providing me with all the required resources. I would like to extend special thanks to all my colleagues for their help and constant support.

This work cannot be completed without the help of above-mentioned person.

//////////

AKTU Roll Number: 200000000000

MCA-2nd Semester

Session: 2025-26

(BATCH: 2024-2026)



JAGRAN INSTITUTE OF MANAGEMENT

An Institute approved by AICTE, Ministry of HRD, Govt. of India & Affiliated to
Dr. A.P.J. Abdul Kalam Technical University, Lucknow, (College Code : 434)



Certificate

This is to certify that the SOFTWARE ENGINEERING LAB (BMC-352)
file has been carried out by the
-----, **MCA – 3rd semester**
Student of Jagran Institute of Management, Kanpur,
under my guidance and supervision in partial fulfillment
of the Master of Computer Application (MCA -3rd semester) during the
Academic Session: 2025-2026.

Mr. Ashish Mishra
(ASSISTANT PROFESSOR)
DEPARTMENT OF MCA

Jagran Institute of Management
Kanpur

Index

S. No.	Title	Page No.	Sign/Remarks
1	What is DFD? Draw a detail First Level DFD for ATM Cash withdrawal process.	01-03	
2	What is SRS? Prepare an SRS document of “Library Management System”, in line with the IEEE recommended standards.	04-09	
3	What is ER Diagram? Create an ER Diagram of “Library Management System”.	10-12	
4	Draw the use case diagram of “Library Management System” and specify the role of each of the actors.	13-14	
5	Prepare state the precondition; post condition and function of each use case.	15	
6	What is class diagram? Identify the classes in “Library Management System” and draw the class diagram.	16-19	
7	Draw the sequence diagram for issue book and return book scenarios.	20-21	
8	What is Activity diagram? Draw the activity diagram for cash withdrawal process from ATM.	22-24	
9	What is component diagram? Draw the component diagram Railway Management System.	25-27	
10	What is State Chart Diagram? Draw the state chart diagram for ATM.	28-30	

Teacher's Signature

()

Q.1) What is DFD? Draw a detail First Level DFD for ATM Cash withdrawal process.

What is DFD?

Data flow diagram is graphical representation of flow of data in an information system. It is capable of depicting incoming data flow, outgoing data flow and stored data. The DFD does not mention anything about how data flows through the system. There is a prominent difference between DFD and Flowchart.

Components:

Entities: Entities include source and destination of the data. Entities are represented by rectangle with their corresponding names

Process: The tasks performed on the data is known as process. Process is represented by circle. Somewhere round edge rectangles are also used to represent process.

Data Storage: Data storage includes the database of the system. It is represented by rectangle with both smaller sides missing or in other words within two parallel lines.

Data Flow: The movement of data in the system is known as data flow. It is represented with the help of arrow. The tail of the arrow is source and the head of the arrow is destination.

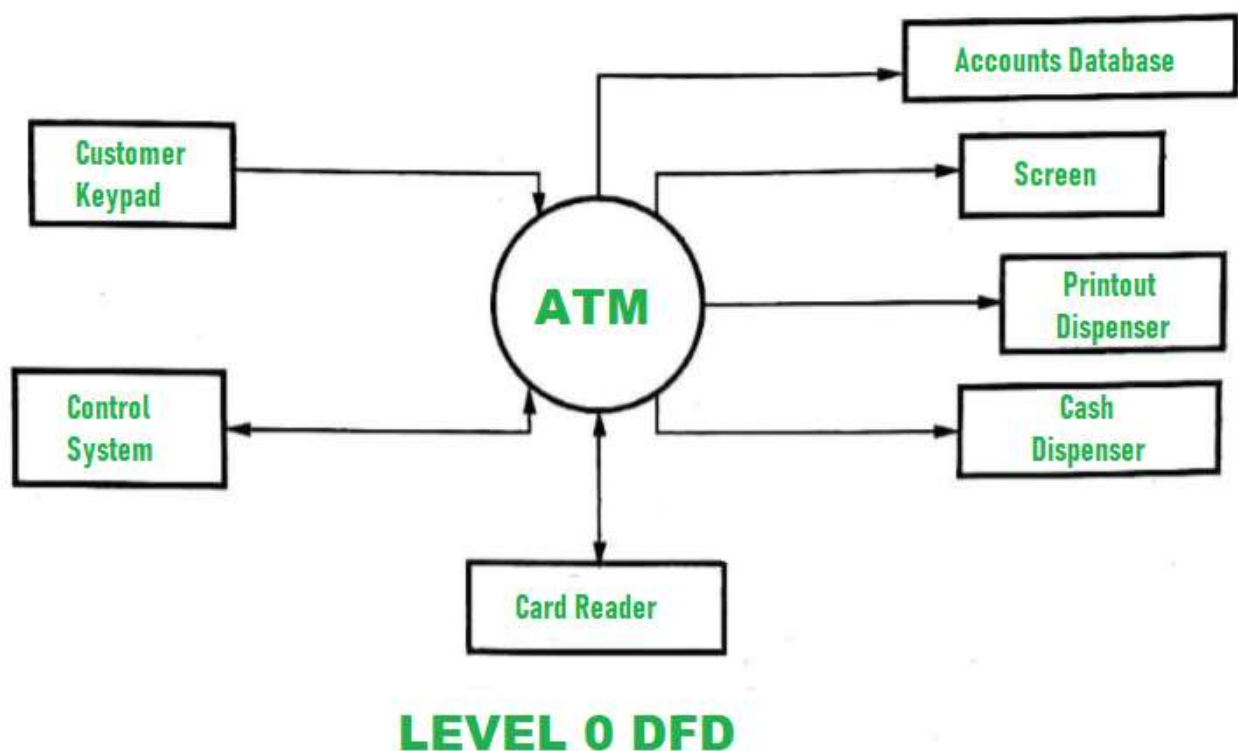
DFD Rules:

- Each process should have at least one input and an output.
- Each data store should have at least one data flow in and one data flow out.
- Data stored in a system must go through a process.
- All processes in a DFD go to another process or a data store

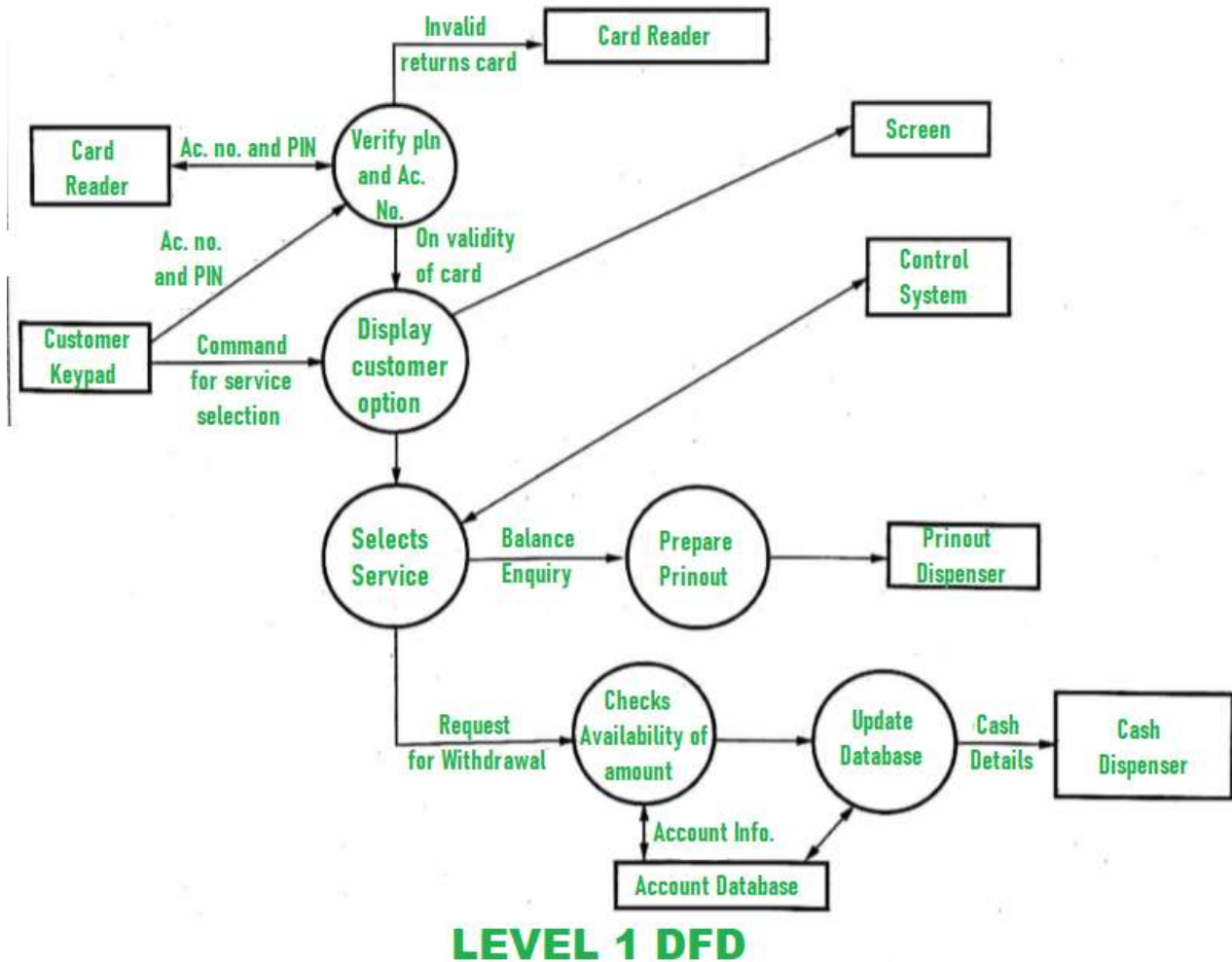
DFD Levels:

A data flow diagram can dive into progressively more detail by using levels and layers, zeroing in on a particular piece. DFD levels are numbered 0, 1 or 2, and occasionally go to even Level 3 or beyond.

DFD Level 0 is also called a Context Diagram. It's a basic overview of the whole system or process being analyzed or modeled. It's designed to be an at-a-glance view, showing the system as a single high-level process, with its relationship to external entities. It should be easily understood by a wide audience, including stakeholders, business analysts, data analysts and developers.



DFD Level 1 provides a more detailed breakout of pieces of the Context Level Diagram. You will highlight the main functions carried out by the system, as you break down the high-level process of the Context Diagram into its sub-processes.



These procedures require information such as record of customers, deposits, balance inquiry, credits and withdrawals from which served as the bases for ATM machines and bank admin to manage the ATM system. This type of data is represented by a data store.

With being knowledgeable about the DFID level 1 of the ATM (Automated Teller Machine) System, you will know then it's broadened context terms

In addition to that, this may also serve as your reference on how the inputs or data fed on the system. Then you will be also informed about the outputs that the system gives

These processes shown in the DFD were all based on the concept of ATM (Automated Teller Machine) System.

Q.2) What is SRS? Prepare an SRS document of "Library Management System in line with the IEEE recommended standards.

An **SRS document** (Software Requirements Specification) is a comprehensive description of the intended purpose and behavior of a software system. It provides a detailed outline of the software's functional and non-functional requirements, as well as constraints and design considerations that the software must meet. The SRS document serves as a blueprint for both developers and stakeholders, ensuring that there is a shared understanding of the project's scope, goals, and expectations.

Key Components of an SRS Document:

1. Introduction:

- **Purpose:** This section explains the purpose of the SRS and who its intended audience is. It often includes a brief overview of the software system.
- **Scope:** Defines the boundaries of the system, including what the software will do and what it will not do.
- **Definitions, Acronyms, and Abbreviations:** A list of terms, acronyms, and definitions that will be used throughout the document.
- **References:** Any external documents or resources that are referenced within the SRS.
- **Overview:** A brief summary of the content structure of the SRS.

2. Overall Description:

- **Product Perspective:** Describes how the software fits within the larger context, such as its relationship with other systems, hardware, or environments.
- **Product Features:** A high-level overview of the key functionalities that the software will provide.
- **User Characteristics:** Details about the intended users of the software, their skills, experience, and the environments they will use the system in.
- **Assumptions and Dependencies:** Any assumptions the project is making, as well as external factors or dependencies that could affect the development or functionality of the system.

3. System Features (Functional Requirements):

This section provides a detailed description of each system feature or functionality.

For each feature, you typically include:

- **Description:** A brief description of the feature.
- **Functional Requirements:** Specific, detailed functionality that the software must provide. These are often written as "the system shall..." statements.
- **Inputs and Outputs:** The inputs required by the feature and the outputs it produces.
- **Behavioral Requirements:** How the feature should behave in certain conditions (e.g., error)

4. External Interface Requirements:

This section defines how the software will interact with other systems, hardware, or external components.

- **User Interfaces:** Descriptions of any graphical user interfaces (GUIs) or command-line interfaces (CLIs), including user interaction flows and screen layouts.
- **Hardware Interfaces:** If the software needs to interface with specific hardware devices, this section defines those interactions.
- **Software Interfaces:** Describes interactions with other software systems, APIs, or services.
- **Communication Interfaces:** Defines how the software will communicate with other systems or networks, including protocols and formats.

5. System Attributes (Non-functional Requirements):

Non-functional requirements are constraints or qualities that the system must meet. These are often related to performance, security, reliability, and scalability.

1. **Performance:** Speed, response time, and efficiency requirements.
2. **Security:** Authentication, authorization, and encryption requirements.
3. **Reliability:** Uptime, error rates, and failure recovery.
4. **Scalability:** The system's ability to handle increased loads.
5. **Maintainability:** How easy it is to update and modify the system.
6. **Usability:** How user-friendly the system needs to be.
7. **Portability:** The system's ability to operate on different platforms or environments.

6. Other Requirements:

This section includes any additional requirements that do not fit into the previous categories. It might include legal, regulatory, or compliance requirements, or any other domain-specific needs.

Benefits of an SRS Document:

1. **Clear Communication:** Ensures that both developers and stakeholders have a shared understanding of what the software will do.
2. **Foundation for Design and Development:** Provides a structured foundation for system design and development.
3. **Basis for Testing:** Helps define the criteria against which the software will be tested.
4. **Project Management:** Assists project managers in estimating resources, timelines, and milestones.
5. **Changes and Maintenance:** When modifications or updates are needed, the SRS helps guide the necessary changes.

SRS Formats:

The IEEE 830-1998 standard is often used as a reference for SRS documents, but companies may adapt it to meet their specific needs. You might encounter different templates or structures, depending on the software development methodology being used (Agile, Waterfall, etc.).

A Software Requirements Specification (SRS) document for a Library Management System (LMS), structured according to the guidelines of the **IEEE Std 830-1998 standard** is given below. This document provides a detailed outline and examples for each section.

Software Requirements Specification for Library Management System

Version 1.0 approved

Prepared by: Team Name/Organization

Date created: December 4, 2025

Table of Contents

1. **Introduction**
 - 1.1 Purpose
 - 1.2 Document Conventions
 - 1.3 Intended Audience and Reading Suggestions
 - 1.4 Product Scope
 - 1.5 References
 - 1.6 Definitions, Acronyms, and Abbreviations
2. **Overall Description**
 - 2.1 Product Perspective
 - 2.2 Product Functions
 - 2.3 User Classes and Characteristics
 - 2.4 Operating Environment
 - 2.5 Design and Implementation Constraints
 - 2.6 Assumptions and Dependencies
3. **Specific Requirements**
 - 3.1 External Interface Requirements
 - 3.2 Functional Requirements
 - 3.3 Non-Functional Requirements
 - 3.4 Software Quality Attributes
4. **Appendices (Optional)**
 - 4.1 Analysis Models (e.g., Use Case Diagrams, ER Diagrams)
 - 4.2 To Be Determined List

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Library Management System (LMS). It will serve as the foundational agreement between the customer (library administration) and the development team, outlining the functional and non-functional requirements that the final product must satisfy.

1.2 Document Conventions

Standard IEEE conventions are used. Key terms such as "User," "Admin," and "Librarian" are capitalized throughout the document. Requirements are formatted as **REQ-[module]-[id]** for clear traceability.

1.3 Intended Audience and Reading Suggestions

The primary audience includes the customer, project managers, software developers, testers, and documentation writers. The document is structured logically, starting with high-level descriptions and moving into specific, detailed requirements in Section 3.

1.4 Product Scope

The LMS is a web-based application designed to automate core library operations, including book cataloguing, member registration, book borrowing/returning, and overdue fine calculation. The system will replace the current manual, paper-based tracking methods.

1.5 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- Existing library procedure manuals (as provided by the client).

1.6 Definitions, Acronyms, and Abbreviations

- **Admin:** The super-user responsible for system configuration.
- **Librarian:** Library staff member managing daily operations (e.g., issuing/returning books).
- **Patron/User:** Library members (students, faculty) who borrow books.
- **ISBN:** International Standard Book Number.
- **LMS:** Library Management System.
- **SRS:** Software Requirements Specification.

2. Overall Description

2.1 Product Perspective

The LMS is a self-contained, standalone web application. It is the primary software system for managing the library's resources and member interactions. It will interface with a relational database (e.g., MySQL) for data storage and retrieval.

2.2 Product Functions

The major functions the system will perform include:

- User/Librarian login and authentication.
- Managing book catalog details (add, edit, delete).
- Managing member registration and accounts.
- Processing book issuance and returns.
- Searching for books by title, author, or ISBN.
- Calculating and tracking overdue fines.
- Generating various library reports (e.g., popular books, overdue list).

2.3 User Classes and Characteristics

- **Patrons (Users):** Expected to have basic computer literacy. They need an intuitive interface to search for books and view their account status.
- **Librarians:** Library staff with moderate computer skills. They require robust administrative interfaces for managing inventory, member records, and transactions efficiently.
- **Admin:** The system administrator has full control over all system functionalities, including user role management and system settings.

2.4 Operating Environment

The system will operate in a web-based environment. It should be compatible with standard web browsers (Chrome, Firefox, Safari) and require a minimum internet connectivity speed of 512 Kbps.

for optimal performance. The backend will run on a Linux-based server using an open-source database technology like MySQL.

2.5 Design and Implementation Constraints

- The system must use an open-source database (e.g., MySQL).
- The user interface design must be responsive and follow the library's branding guidelines.
- The system must adhere to all local data privacy regulations regarding member information.

2.6 Assumptions and Dependencies

- It is assumed that users will have reliable internet access to use the web interface.
- The system relies on the accurate and timely input of initial book inventory data by library staff.

3. Specific Requirements

3.1 External Interface Requirements3.1.1 User Interface

The system shall provide a graphical user interface (GUI) accessible via a web browser. The interface will include forms, standard buttons, and navigation menus. Key screens include the Login page, Search page, User Dashboard, and Admin Dashboard.

3.1.2 Software Interfaces

- **Database:** The system shall interface with a MySQL database to store and retrieve all library data.
- **Operating System:** The server-side software will run on a Windows or Linux-based operating system.

3.2 Functional Requirements

Functional requirements are grouped by user class and feature.

3.2.1 Module: User Authentication

- **REQ-AUTH-1:** The system shall allow both Patrons and Librarians to log in using a unique ID and password.
- **REQ-AUTH-2:** The system shall validate login credentials within 3 seconds under normal network conditions.
- **REQ-AUTH-3:** The system shall provide a password reset mechanism via email link.

3.2.2 Module: Book Management (Librarian/Admin)

- **REQ-BOOK-1:** The system shall allow Librarians to add new book records with details like ISBN, Title, Author(s), Genre, and Quantity.
- **REQ-BOOK-2:** The system shall prevent duplicate ISBN entries and alert the Librarian if an ISBN already exists.
- **REQ-BOOK-3:** The system shall allow Librarians to update book information, including the current stock status.
- **REQ-BOOK-4:** The system shall allow Librarians to search the book catalogue using multiple criteria (title, author, ISBN).

3.2.3 Module: Membership Management (Librarian/Admin)

- **REQ-MEM-1:** The system shall allow Librarians to register new Patrons, capturing name, contact details, ID number, and email.
- **REQ-MEM-2:** The system shall allow Librarians to view and edit Patron details and membership status.
- **REQ-MEM-3:** The system shall allow Librarians to track the history of books borrowed by a

specific Patron.

3.2.4 Module: Book Circulation (Librarian)

- **REQ-CIRC-1:** The system shall allow a Librarian to issue a book to a Patron by scanning/entering the book's ID and the Patron's ID.
- **REQ-CIRC-2:** The system shall automatically record the issue date and calculate the due date (default: 14 days).
- **REQ-CIRC-3:** The system shall allow a Librarian to record a book return.
- **REQ-CIRC-4:** The system shall automatically calculate a fine if the return date is past the due date (rate: \$1 per overdue day).
- **REQ-CIRC-5:** The system shall prevent a Patron from borrowing more than 3 books at a time.

3.2.5 Module: Patron Features (Patron)

- **REQ-PAT-1:** The system shall allow Patrons to search the book catalogue by title, author, or keyword.
- **REQ-PAT-2:** The system shall display the real-time availability status of a book ("Available", "On Loan", "Reserved").
- **REQ-PAT-3:** The system shall allow Patrons to view their current borrowed books, due dates, and outstanding fines.

3.3 Non-Functional Requirements

3.3.1 Performance Requirements

- **NFR-PERF-1:** The system shall load all major pages (Login, Dashboard, Search Results) within 5 seconds under normal load conditions (up to 50 concurrent users).
- **NFR-PERF-2:** Database queries for book searches or user information retrieval shall complete within 3 seconds.

3.3.2 Safety and Security Requirements

- **NFR-SEC-1:** All sensitive data (passwords, PII) shall be stored in an encrypted format in the database.
- **NFR-SEC-2:** The system shall prevent unauthorized access by implementing strong authentication mechanisms.
- **NFR-SEC-3:** The system shall maintain audit logs for all administrative actions (adding/deleting books or members).
- **NFR-SEC-4:** A daily database backup mechanism must be implemented.

3.3.3 Software Quality Attributes

- **Usability:** The system should be intuitive for Patrons with basic computer knowledge, requiring minimal training.
- **Reliability:** The system should have an uptime of 99.9% during library operating hours.
- **Maintainability:** The software should be developed using modular code and clear documentation to facilitate future updates and maintenance.

4. Appendices (Optional)

4.1 Analysis Models

This section would typically include visual diagrams such as Use Case Diagrams, Class Diagrams, or Entity-Relationship (ER) Diagrams that illustrate the system's structure and behaviour.

4.2 To Be Determined List

- The exact fine rate for overdue books needs final confirmation from the library board.
- Integration with the university's existing student ID card system is a potential future requirement.

Q.3) What is ER Diagram? Create an ER Diagram of “Library Management System”.

An **Entity-Relationship (ER) Diagram** is a graphical representation of the **entities** (objects or concepts) within a system and the **relationships** between those entities. It is widely used in database design and systems analysis to visually model the structure of data and its interconnections. The ER diagram serves as a blueprint for how the data will be stored and manipulated within a database system.

Components of an ER Diagram:

1. Entities:

- **Definition:** An entity represents a real-world object, concept, or thing within the system that has distinct existence.
- **Symbol:** Entities are represented by **rectangles** in an ER diagram.

Entity Types:

1. **Strong Entity:** Has a unique identifier or primary key. It doesn't rely on another entity to exist. For example, a **Student** entity with a unique student ID.
2. **Weak Entity:** Cannot exist without a strong entity. It relies on a parent entity for identification. For example, a Dependent entity (e.g., a child) might depend on an Employee entity.

2. Attributes:

- **Definition:** An attribute is a property or characteristic of an entity that helps describe it. For example, a Student entity might have attributes like Student ID, Name, Email, Date of Birth, etc.
- **Symbol:** Attributes are typically represented by **ovals** connected to the entity.

Types of Attributes

1. **Simple Attribute:** Cannot be divided further (e.g., Student Name).
2. **Composite Attribute:** Can be divided into smaller sub-parts (e.g., Full Name might be divided into First Name and Last Name).
3. **Derived Attribute:** Its value can be derived from other attributes (e.g., Age derived from Date of Birth).
4. **Multi-valued Attribute:** Can have multiple values for a single entity (e.g., a Phone Numbers attribute for an Employee entity, where an employee might have more than one phone number).

3. Relationships:

- **Definition:** A relationship describes how two or more entities are related to each other. For example, a Student might Enroll in a Course, or a Professor might Teach a Course.
- **Symbol:** Relationships are represented by diamonds in an ER diagram.

Types of Relationships

- a. **One-to-One (1:1):** One instance of an entity is related to one instance of another entity. Example: A Passport is assigned to only one Citizen, and each Citizen has only one Passport.
- b. **One-to-Many (1:N):** One instance of an entity is related to multiple instances of another entity. Example: A Professor can teach multiple Courses, but each Course is taught by only one Professor.
- c. **Many-to-Many (M:N):** Multiple instances of one entity can be related to multiple instances of another entity. Example: Students can enroll in multiple Courses, and each Course can have multiple Students enrolled.

4. Cardinality:

Cardinality defines the number of instances of one entity that can or must be associated with each instance of another entity. Cardinalities are typically represented using symbols near the relationship lines:

- **One:** Represented by a single
- **Many:** Represented by a "crow's foot", indicating that one instance of an entity can be related to multiple instances of another entity.
- **Optional:** Indicated by a circle, meaning that an instance of an entity may or may not be related to instances of another entity.

5. Primary Keys and Foreign Keys:

- **Primary Key:** A unique identifier for each instance of an entity. In the Student entity, for instance, the Student ID might be the primary key.
- **Foreign Key:** An attribute that links one entity to another. For instance, in a Course Enrollment relationship, the Student ID might be a foreign key linking the Student entity to the Course entity.

6. Weak Entities and Identifying Relationships:

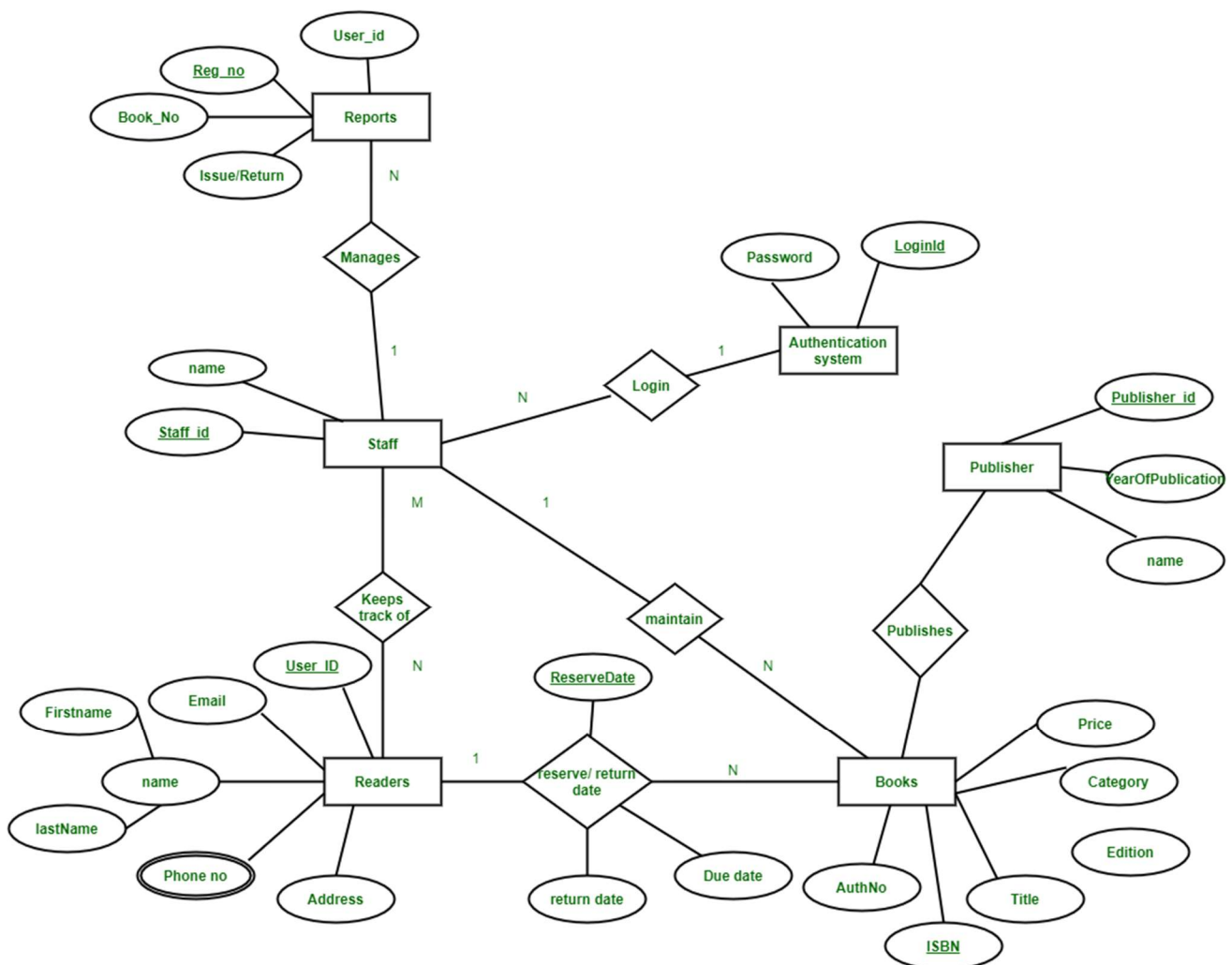
- **Weak Entity:** As mentioned earlier, a weak entity cannot exist without a corresponding strong entity. Weak entities are represented with a double rectangle.
- **Identifying Relationship:** This type of relationship is used to link a weak entity to its strong entity. It is represented with a double diamond.

Types of ER Diagrams:

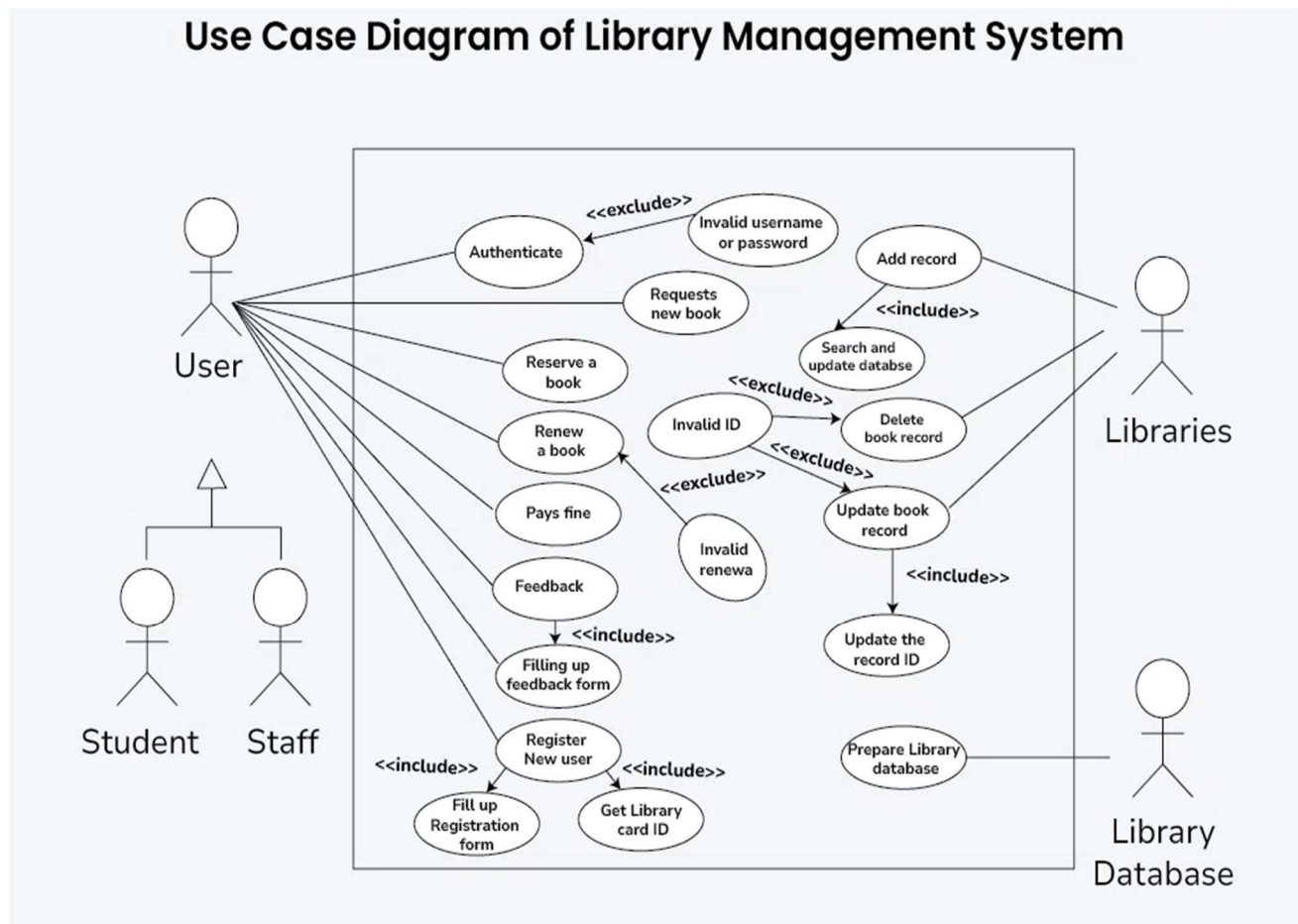
1. **Chen's Notation:** This is the most widely used form of ER diagrams, named after Peter Chen, who introduced the ER model. It uses rectangles for entities, ovals for attributes, and diamonds for relationships.
2. **Crow's Foot Notation:** A more modern approach to ER diagrams that uses "crow's foot" symbols to denote cardinality. This is often easier to read, especially for more complex systems.
3. **UML (Unified Modeling Language):** While UML class diagrams are technically different from ER diagrams, they are often used in similar contexts. They represent classes and relationships in object-oriented systems and can include similar elements like entities, attributes, and relationships.

Best Practices:

1. **Keep it Simple:** Avoid overcrowding the diagram with too many entities and relationships at once. Break it down into smaller diagrams if needed.
2. **Use Consistent Notation:** Stick to one type of notation (e.g., Chen's or Crow's Foot) to ensure clarity and consistency.
3. **Indicate Cardinality Clearly:** Make sure the cardinality is well-defined to avoid confusion in how entities relate to each other.



Q.4) Draw the use case diagram of “Library Management System” and specify the role of each of the actors.



In a "Library Management System" use case diagram, the primary actors are the Librarian and the **Member/User**. The Librarian's role is to manage books, users, and library operations, such as adding, updating, and deleting books and user records, issuing books, and managing fines. The Member's role is to interact with the system for personal use, including searching for books, borrowing, returning, and checking their account status.

Role of each actor

1. Librarian

- Manages the library's collection by adding, updating, and deleting books and their information.
- Handles user registrations and manages user accounts.
- Issues books to members.
- Processes book returns.
- Manages fines, calculates them, and processes payments.
- Maintains library records and generates reports.

2. Member/User

- Registers with the system to get a library card.

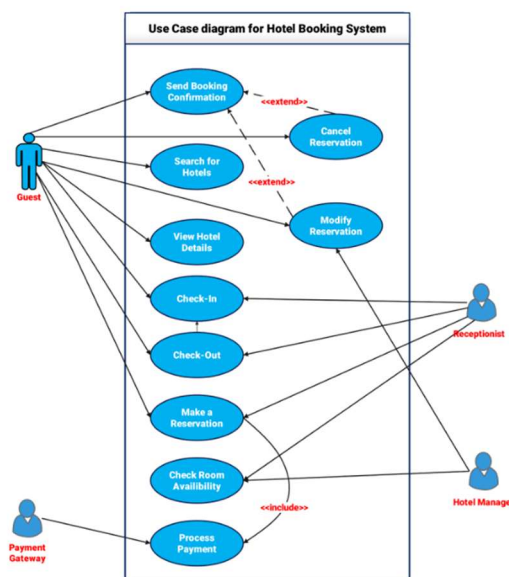
- Searches for books and checks their availability.
- Borrows books.
- Returns books.
- Checks their account status, including borrowed books and due dates.
- Pays fines.
- Can renew borrowed books.
- Can provide feedback on books.

Q.5) Prepare state the precondition; post condition and function of each use case.

A use case's precondition is the required state of the system before the use case can begin. The postcondition is the guaranteed state of the system after the use case completes successfully. The function is the main goal of the use case, expressed as a step-by-step narrative of what the user and system do.

Use case breakdown:

Component	Description	Example: "Book a Hotel Room"
Precondition	The conditions that must be true for the use case to start.	1. User is logged into their account. 2. User has a valid payment method on file.
Function	The steps the actor and system take to achieve the use case's goal.	1. User enters search criteria (dates, location). 2. System retrieves and displays available rooms. 3. User selects a room. 4. System confirms the booking and charges the payment method.
Postcondition	The state of the system after the use case is successfully completed.	1. A hotel reservation is created in the system. 2. The user's payment method is charged. 3. A confirmation email is sent to the user.



Q.6) What is class diagram? Identify the classes in “Library Management System” and draw the class diagram.

A **Class Diagram** is one of the most fundamental components of **Unified Modeling Language (UML)** and is a key tool in object-oriented software design. It provides a static view of the system by modeling the system's classes, their attributes, methods (operations), and the relationships between the classes. Class diagrams are essential for visualizing the structure of an application, offering a blueprint for developers to understand how the different parts of the system interact and evolve.

In simpler terms, a class diagram depicts how classes in an object-oriented system are organized and how they relate to each other. It is one of the most widely used diagrams in UML and is an essential part of object-oriented design because it helps developers and stakeholders understand the design of the system before coding begins.

Components of a Class Diagram:

1. Classes:

A **class** in a class diagram represents a blueprint for creating objects (instances). Each class encapsulates data (attributes) and behavior (methods or operations). For example, in a Library Management System, a `Book` class could have attributes like `title` and `author`, and methods like `checkout()` or `returnBook()`.

A class is typically represented by a rectangle divided into three sections:

- a. **Top Section:** Contains the class name.
- b. **Middle Section:** Lists the attributes(data fields) of the class.
- c. **Bottom Section:** Lists the methods or operations of the class.

2. Attributes:

Attribute represent the data or properties associated with a class. They define the state of the object. For example, a `Person` class might have attributes like `name`, `age`, and `address`.

In a class diagram, attributes are listed in the middle section of the class rectangle and are typically written as `visibility name : type` (e.g., `+ age: int`), where:

Visibility (`+` for public, `-` for private, `#` for protected).

Name is the attribute's identifier (e.g., `age`).

Type is the data type of the attribute (e.g., `int`, `String`).

3. Methods (Operations):

Methods define the behavior of a class. They represent actions or functions that can be performed on the object or by the object. For instance, a `BankAccount` class might have methods like `deposit()` or `withdraw()`. Methods are listed in the bottom section of the class rectangle, and the typical syntax is `visibility name(parameters) : returnType` (e.g., `+ deposit(amount: double): void`).

4. Relationships Between Classes:

Class diagrams also model the relationships between classes. These relationships help to describe how classes interact with each other. Common relationships include:

- **Association:** A general relationship between two classes, often represented by a solid line. It can be one-to-one, one-to-many, or many-to-many.
- **Inheritance:** Also known as generalization, inheritance represents an "is-a" relationship where one class inherits attributes and methods from another. This is shown with a line and a hollow triangle pointing towards the parent class.
- **Aggregation:** A type of "whole-part" relationship where the part can exist independently of the whole. It is depicted with a line and an open diamond at the whole end.
- **Composition:** A stronger form of aggregation where the part cannot exist without the whole. It is represented with a solid diamond at the whole end.
- **Realization:** This shows that a class implements an interface. It is represented by a dashed line with a hollow triangle pointing towards the interface.

5. Visibility Modifiers:

Visibility modifiers define the scope of access to attributes and methods:

Public (+): The attribute or method is accessible by any class.

Private (-): The attribute or method is accessible only within the class itself.

Protected (#): The attribute or method is accessible within the class and by subclasses.

Package (~): The attribute or method is accessible only within the same package.

Benefits of Class Diagrams:

1. Clear Representation of System Structure: Class diagrams provide a clear and organized visual representation of the system's structure, making it easier for developers, designers, and stakeholders to understand the design.

2. Facilitates Object-Oriented Design: By showing the relationships and interactions between different classes, class diagrams are fundamental in designing object-oriented systems. They help identify how classes will work together and interact.

3. Helps in Code Generation: Many development tools allow you to generate code skeletons directly from class diagrams. This ensures that the design can be directly translated into the implementation phase with minimal errors.

4. Improved Communication: Class diagrams serve as a common language between technical and non-technical stakeholders. They allow business analysts, designers, and developers to discuss and refine system architecture more effectively.

5. Documentation: Class diagrams serve as a permanent, visual record of the system's design, which is essential for maintaining and evolving the system over time.

Class diagrams are essential tools for visualizing, documenting, and designing object-oriented systems. They provide a structured way to represent the key components of a system, including classes, attributes, methods, and relationships. By helping to clarify system design, class diagrams not only make it easier for developers to understand the structure of the software but also improve

communication among team members and stakeholders.

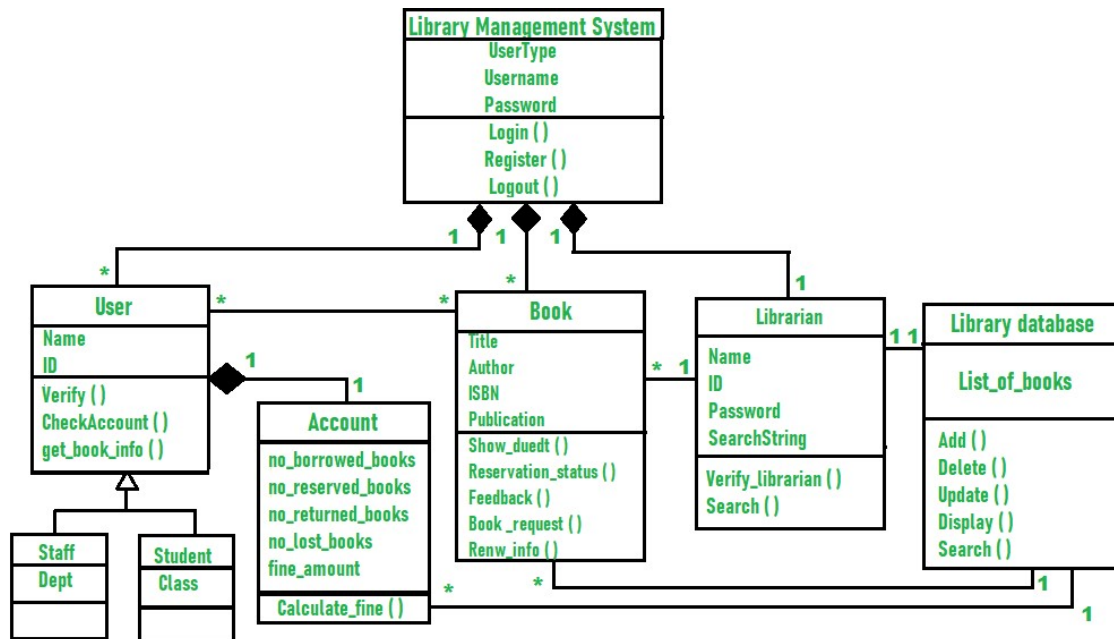
A class diagram for a Library Management System includes classes like Book, Member/User, and Librarian, with each class having attributes (data) and methods (functions). Key relationships include aggregation (e.g., Library "has-a" Book), inheritance (e.g., Librarian "is-a" User), and associations (e.g., a User "borrows" a Book). Other potential classes include Transaction, Fine, and Inventory.

Core classes:

- **User / Member:**
 - **Attributes:** User ID, name, contact information (email, phone), address, account status.
 - **Methods:** register(), login(), logout(), searchBook(), borrowBook(), returnBook(), updateProfile().
- **Book:**
 - **Attributes:** Book ID, title, author, ISBN, publication date, genre, availability status.
 - **Methods:** addBook(), removeBook(), updateBookDetails().
- **Librarian:**
 - **Attributes:** Librarian ID, name, contact information.
 - **Methods:** addMember(), removeMember(), searchMember(), issueBook() (to a user), returnBook() (from a user), manageInventory().

Supporting classes:

- **Transaction:**
 - **Attributes:** Transaction ID, User, Book, issue date, due date, return date.
 - **Methods:** createTransaction(), calculateFine().
- **Fine:**
 - **Attributes:** Fine ID, amount, status (paid/unpaid), Transaction ID.
 - **Methods:** calculateFine(), collectFine(), waiveFine().
- **Author:**
 - **Attributes:** Author ID, name, bio.
 - **Methods:** writeBook(), publishBook().
- **Inventory:**
 - **Attributes:** List of all books, current status of each book.
 - **Methods:** checkBookAvailability(), updateBookStatus().
- **Notification:**
 - **Attributes:** User, message type (e.g., overdue, new arrival).
 - **Methods:** sendNotification().



CLASS DIAGRAM FOR LIBRARY MANAGEMENT SYSTEM

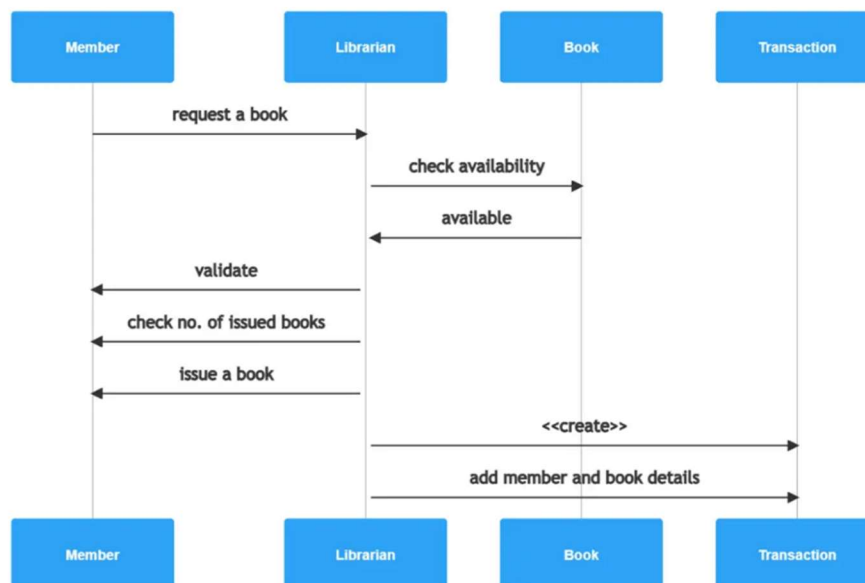
Q.7) Draw the sequence diagram for issue book and return book scenarios.

A sequence diagram for issuing a book shows a member requesting a book from a librarian, who then validates the book's availability and the member's eligibility before recording the transaction and updating the book's status. A return book sequence diagram involves a member providing a book to the librarian, who checks for overdue fines, updates the book's status to available, and records the return in the transaction and member record.

Issue Book Scenario

Objects: Member, Librarian, Book, Transaction

1. Request Book: The Member sends a message to the Librarian.
2. Check Availability: The Librarian checks the Book's availability via a message.
3. Validate Member: The Librarian sends messages to the Member.
4. Confirm and Update: If validation succeeds, the Librarian sends a message to the Book, creating a new Transaction entry via a message, and updating the Member's record.
5. Acknowledge: The Librarian sends a success message back to the Member.

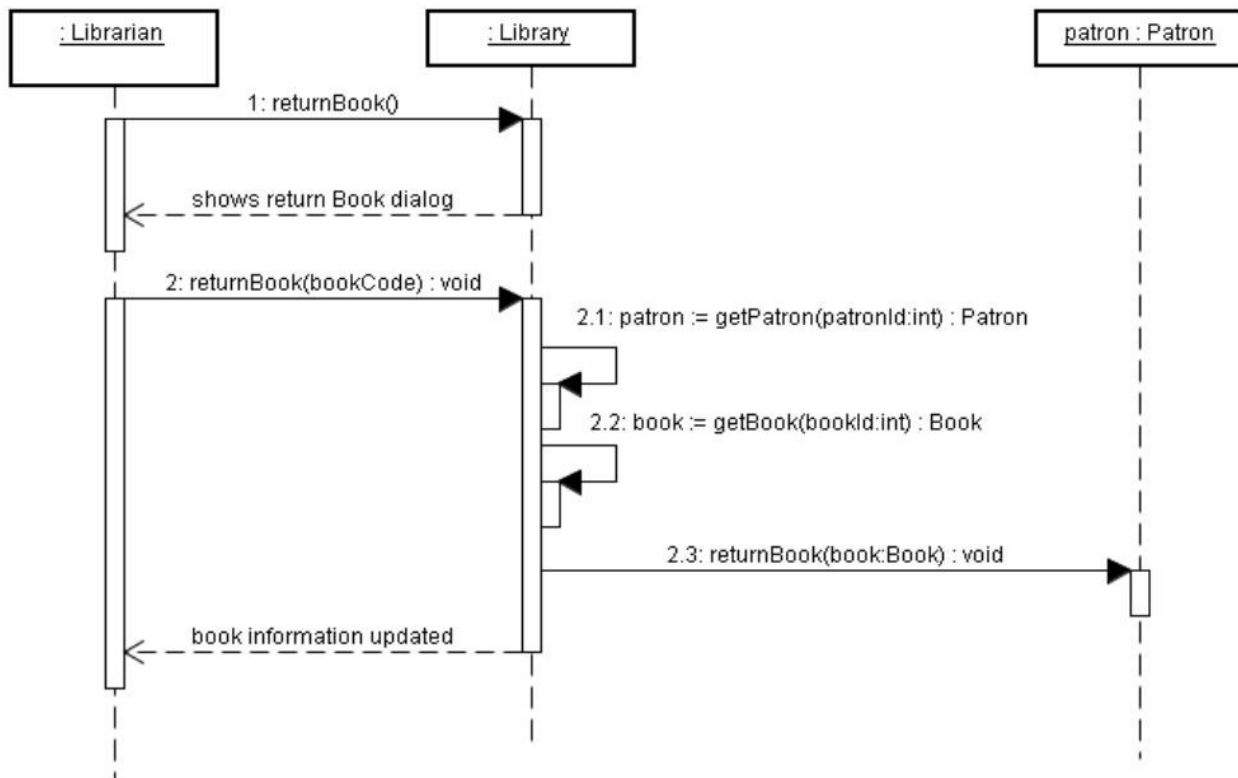


Return Book Scenario

Objects: Member, Librarian, Book, Transaction

1. Return Book: The Member sends a message to the Librarian.
2. Check Fines (Optional): The Librarian checks if the Member has any overdue fines.
3. Update Book Status: The Librarian sends a message to the Book, changing it to 'available'.
4. Update Member Record: The Librarian sends a message to the Transaction to reflect the return, possibly updating the Member's record.
5. Acknowledge: The Librarian sends a success message back to the Member.

Sequence Diagram – Return Book



Q.8) What is Activity diagram? Draw the activity diagram for cash withdrawal process from ATM.

An **Activity Diagram** is a type of behavioral diagram used in the **Unified Modeling Language (UML)** to represent workflows, processes, or the dynamic aspects of a system. It visually describes the sequence of activities or steps involved in a particular process, showing how an operation or function flows from one step to another. Activity diagrams are typically used to model the business processes, user interactions, and the internal workflows of a system, making them an essential tool in both software design and process management.

Components of an Activity Diagram:

1. Activities:

- **Definition:** Activities represent the individual steps or actions in a process. Each activity is an action or task that is executed within the workflow.
- **Symbol:** Activities are depicted as rounded rectangles or ovals in the diagram.
- **Example:** In an Online Shopping System, activities could include Login, Browse Products, Add to Cart, and Checkout.

2. Transitions:

- **Definition:** A transition represents the flow from one activity to the next. It shows the direction of the workflow.
- **Symbol:** Transitions are represented by arrows that connect activities.
- **Example:** After a user completes the Login activity, the flow might transition to Browse Products.

3. Initial Node:

- **Definition:** The initial node represents the starting point of the activity diagram. It indicates where the process begins.
- **Symbol:** The initial node is represented by a solid black circle.
- **Example:** The Login Page in an online shopping system could be the initial step where the user starts the workflow.

4. Final Node:

- **Definition:** The final node signifies the end of the activity or the process.
- **Symbol:** The final node is represented by a solid circle with a border.
- **Example:** In the Checkout process of an online shopping system, the final node might represent the Order Confirmation step, indicating the end of the shopping transaction.

5. Decision Node:

- **Definition:** A decision node represents a branching point in the process, where the flow splits based on a condition or decision.

- **Symbol:** It is represented by a diamond shape.
- **Example:** After the Checkout activity, a decision node could branch based on whether the payment is successful or failed.

6. Merge Node:

- **Definition:** A merge node is used to bring together multiple alternate flows in the process. It combines different paths into a single one.
- **Symbol:** Similar to the decision node, it is also represented by a diamond shape, but the flow is coming from multiple directions.

7. Fork Node:

- **Definition:** A fork node splits the flow of control into multiple concurrent activities. This is useful when parallel tasks need to be performed.
- **Symbol:** Represented by a horizontal or vertical bar.
- **Example:** After placing an order, the system might send an email confirmation and update the inventory at the same time. These actions are parallel.

8. Join Node:

- **Definition:** A join node brings together multiple parallel flows into one flow. It synchronizes the execution of concurrent activities.
- **Symbol:** Represented by a horizontal or vertical bar, similar to the fork node.

9. Swimlanes:

- **Definition:** Swimlanes divide the activity diagram into distinct sections, each representing a particular actor or role. This helps show who or what is responsible for each activity in the workflow.
- **Symbol:** Swimlanes are represented by vertical or horizontal partitions in the diagram.
- **Example:** In a business process, one swimlane could represent the Customer, another could represent the System, and another the Warehouse.

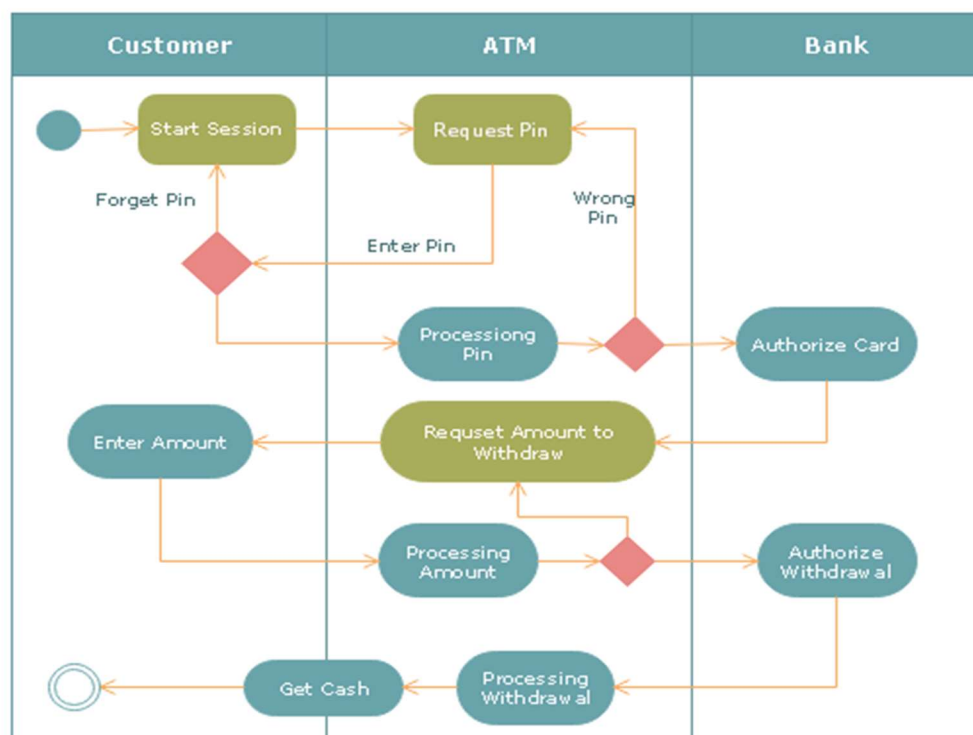
Benefits of Activity Diagrams:

1. **Visualizes Complex Processes:** Activity diagrams are excellent at breaking down complex business workflows or system processes into simple, understandable steps. This makes them an essential tool for both technical and non-technical stakeholders.
2. **Improves Understanding of Workflow:** Activity diagrams help to better understand how a system behaves in response to user actions or business processes, ensuring that each step is clear and logical.
3. **Supports Parallelism and Synchronization:** Activity diagrams are effective in modeling parallel or concurrent activities, helping to represent workflows that involve tasks that need to be performed simultaneously.

4. **Useful in Business Process Modeling:** These diagrams are often used in business process modeling (BPM) to capture processes in organizations and improve workflow efficiency.
5. **Helps in Identifying System Flows:** For software engineers and developers, activity diagrams provide a clear representation of how the system will react to user inputs, helping to align design and implementation.
6. **Facilitates Communication:** Activity diagrams provide a common visual language that helps developers, analysts, designers, and stakeholders communicate more effectively, ensuring everyone is on the same page.

Activity diagrams are a powerful tool in **UML** used to model dynamic aspects of a system. By representing workflows, processes, and decision-making in a system, activity diagrams provide a detailed, easy-to-understand view of how processes unfold, from start to finish. They are essential in **system design, business process modeling, and software development**, helping ensure that processes are optimized, efficient, and correctly implemented.

Activity Diagram for Cash withdrawal process from ATM



Q.9) What is component diagram? Draw the component diagram Railway Management System.

A **Component Diagram** is a type of **structural diagram** in **UML (Unified Modeling Language)** that models the components (or modules) of a software system, their dependencies, and how they interact. It provides a high-level view of the system's architecture by showing how various components work together to form a complete system. The components are typically modular, encapsulated units of functionality, which can be developed, deployed, and maintained independently. Component diagrams are particularly useful in large systems to visualize the architecture, clarify the relationships between parts of the system, and guide development and deployment.

Components of a Component Diagram:

1. Component:

- **Definition:** A component is a modular unit of a system that encapsulates a set of related functions or services. It can be thought of as a "black box" with well-defined interfaces for interaction.
- **Symbol:** Components are represented by rectangles with two smaller rectangles at the top to symbolize the "provided" and "required" interfaces of the component.
- **Example:** A 'PaymentService' component might provide functions for processing payments, while a 'UserAuthentication' component handles user login.

2. Interfaces:

- **Definition:** An interface represents a contract or a set of operations that a component provides or requires. It allows components to communicate without needing to know the internal workings of each other.
- **Symbol:** An interface is represented as a circle (lollipop) for a provided interface and a half-circle (socket) for a required interface.
- **Example:** The 'PaymentService' component might have a provided interface like 'IPaymentGateway', which other components use to interact with it.

3. Dependencies:

- **Definition:** A dependency represents the relationship where one component relies on the functionality of another. If one component uses the functionality of another, it is said to have a dependency on that component.
- **Symbol:** Dependencies are shown by a dashed arrow pointing from the dependent component to the component it depends on.
- **Example:** A 'CheckoutService' component might depend on the 'PaymentService' to complete transactions.

4. Ports:

- **Definition:** A port is a specific point of interaction between a component and its environment.

It represents an entry or exit point through which the component provides or requires services.

- **Symbol:** A port is represented as a small square on the boundary of a component.
- **Example:** A 'Database Component' might have a port through which it provides data access services to other components.

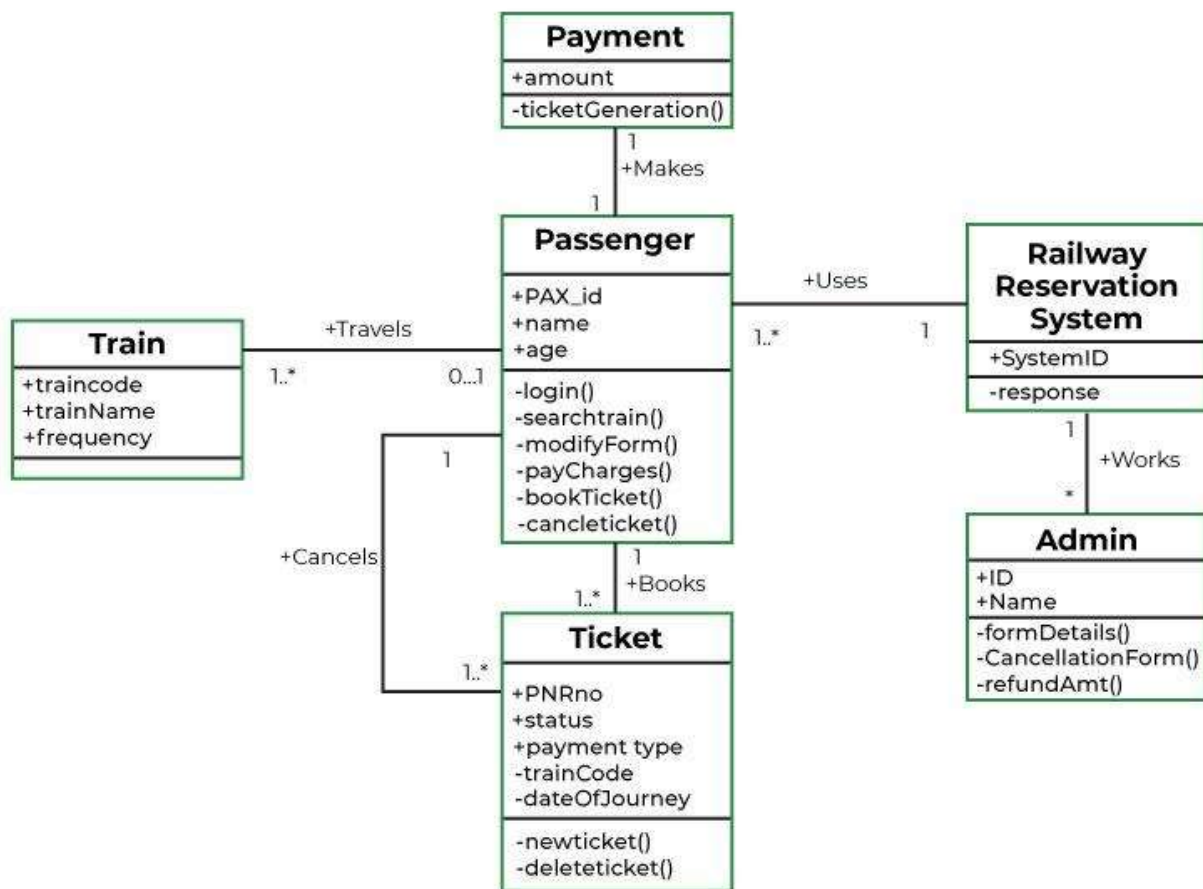
5. Relationships:

- **Associations:** Components are connected through associations, often via interfaces, showing how they communicate with each other.
- **Realization:** A realization relationship is shown between a component and an interface it implements. It indicates that the component fulfills the contract defined by the interface.
- **Usage:** This shows how one component uses services or functions of another component.

Benefits of Component Diagrams:

1. **Clear System Architecture:** Component diagrams provide a high-level, clear visualization of a system's modular architecture. They help to understand how the system is divided into components and how those components interact.
2. **Decouples System Components:** By modeling the system in terms of independent components, component diagrams show how individual pieces of functionality can be developed, deployed, and updated independently, reducing the system's complexity.
3. **Promotes Reusability:** Components often represent reusable pieces of software that can be independently developed and reused in different systems, thereby improving efficiency and reducing development time.
4. **Simplifies Deployment:** Since components can be mapped to separate deployment units, component diagrams help in visualizing the deployment model of the system and understanding how components are distributed across hardware or software platforms.

A **Component Diagram** is a powerful tool for modeling the modular structure of a system. By showing how components interact through interfaces and dependencies, it provides a high-level view of system architecture, helping developers, architects, and stakeholders understand and design complex systems effectively. These diagrams promote modularity, reusability, and scalability in system development, which are essential for maintaining and evolving modern software systems.



Q.10) What is State Chart Diagram? Draw the state chart diagram for ATM.

A **State Chart Diagram**, also known as a **State Machine Diagram** or **State Diagram**, is a type of **behavioral diagram** in UML (**Unified Modeling Language**) that models the dynamic behavior of a system or an object in response to external stimuli. It shows how an object or system transitions between various states based on events and conditions, making it an essential tool for representing the life cycle of an object and its state changes over time.

State chart diagrams are particularly useful in modeling systems where the behavior of entities is highly dependent on their state, such as embedded systems, workflow management systems, and real-time applications. By visualizing the states and transitions, state charts help clarify how objects or systems behave in different contexts.

Key Components of a State Chart Diagram:

1. States:

- **Definition:** A state represents a condition or situation in which an object or system exists during its life cycle. An object may remain in a state until an event triggers a transition to another state.
- **Symbol:** States are represented by rounded rectangles or ovals.
- **Example:** In a traffic light system, states could include Red, Green, and Yellow.

2. Transitions:

- **Definition:** A transition represents the movement from one state to another, triggered by an event. Transitions can occur in response to external events or conditions, often with associated actions.
- **Symbol:** Transitions are represented by arrows connecting states. The arrow points from the source state to the destination state.
- **Example:** In a traffic light system, a transition could be from Red to Green when a timer expires.

3. Events:

- **Definition:** An event is an occurrence that triggers a transition from one state to another. Events can be external (like user input or system signals) or internal (like timeouts or process completions).
- **Symbol:** Events are often labeled near the transition arrows.
- **Example:** In a payment system, the event triggering a state change could be an Authorize Payment action.

4. Actions:

- **Definition:** An action is a process or task performed when a transition occurs or when an object enters or exits a state.
- **Symbol:** Actions are written next to the transition or state where they are executed.

- **Example:** When a traffic light changes from Red to Green, an action could be Turn on Green Light.

5. Initial State:

- **Definition:** The initial state is where the system or object begins its life cycle. This is typically the starting point of a state machine diagram.
- **Symbol:** The initial state is represented by a solid black circle.
- **Example:** In a user authentication system, the initial state could be Login Screen.

6. Final State:

- **Definition:** The final state represents the end of the object's life cycle or the completion of a specific process. Once an object reaches the final state, the state machine is considered complete.
- **Symbol:** The final state is depicted by a solid circle with a border around it.
- **Example:** For a file download process, the final state might be "Download Complete".

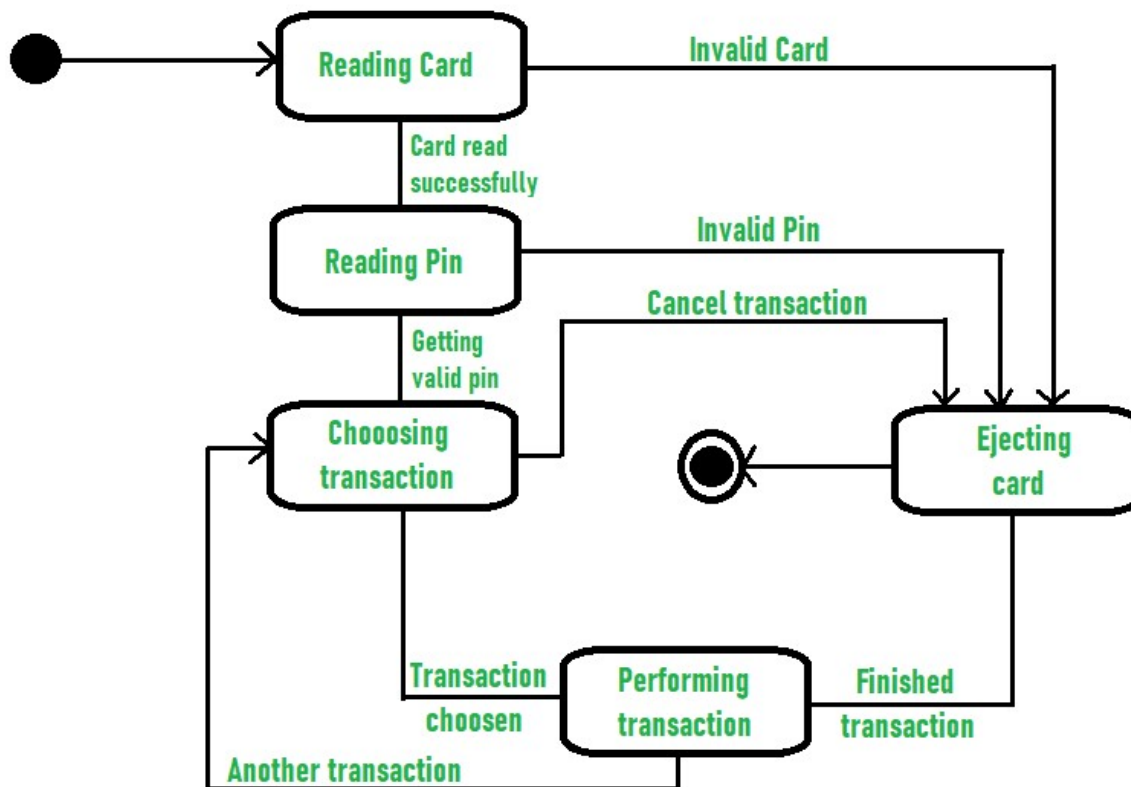
7. State Transitions with Guards:

- **Definition:** A guard is a condition that must be true for a transition to occur. Guards help define more specific rules for when a state change can happen.
- **Symbol:** Guards are often written in square brackets near the transition arrow (e.g. [balance > 0]).
- **Example:** In a bank account system, a guard condition might ensure a transition from Pending Transaction to Completed only when the account balance is sufficient.

Benefits of State Chart Diagrams:

1. **Modeling Complex Behavior:** State charts are particularly useful for systems with complex states and transitions, like embedded systems, workflow management, and interactive applications. They help to visualize how an object behaves in different situations, based on its state.
2. **Improved System Understanding:** By representing all possible states and transitions of a system, state chart diagrams provide a clear understanding of how a system responds to different inputs, making them essential for both designers and developers.
3. **Clear Event Handling:** State diagrams allow you to explicitly show how events trigger state changes, which is particularly useful in real-time systems, user interfaces, or process control applications.
4. **Error Handling and System Recovery:** They help identify possible errors or exceptions by showing failure states and recovery actions, improving the robustness of a system.

A **State Chart Diagram** is a powerful tool for modeling dynamic behavior in systems where the state of an object or system drives its behavior. It helps designers and developers understand the different states that an object can be in, the transitions between those states, and the events that trigger state changes. By providing a clear visual representation of how a system behaves over time, state diagrams are essential for designing, testing, and maintaining systems with complex state-dependent behavior.



State Transition Diagram for ATM System